

智能体新生态和 OpenClaw

智能体型生态 · 从原理到实践

目录概览

- 第一章 Harness Engineering 驾驭工程：理论基础
- 第二章 OpenClaw 是什么：产品架构与核心概念
- 第三章 OpenClaw 工作原理与流程：端到端串讲
- 第四章 OpenClaw 开发指南：Skill 与 MCP 开发
- 第五章 OpenClaw 文件结构详解：开发结果的存放位置
- 第六章 OpenClaw 配置详解：六大类配置说明
- 第七章 行业实践：财务垂直行业智能体开发实例

第一章 Harness Engineering 驾驭工程

1.1 什么是 Harness Engineering

Harness Engineering (驾驭工程) 是面向大模型与 AI 智能体的系统工程方法论。

核心思想：AI Agent = 基础大模型 + Harness 管控层。不改造模型本身，而是给 AI 搭建一套规则约束、流程管控、结果校验、错误闭环的管控体系——像给烈马配上缰绳马具，引导 AI 在业务中可控、合规、稳定、可复现地长期运行。

1.2 为什么需要 Harness

- 原生大模型易幻觉、输出不稳定、行为不可预测；
- 纯提示词 / 上下文工程只能管单次交互，管不住长期复杂任务与多步骤 workflow；
- AI Agent 容易流程跑偏、无限循环、越权操作、产出不合规内容；
- 企业落地需要可审计、可约束、防犯错、可迭代修复的工程化机制，不能靠模型自觉；
- 解决 AI 从 demo 实验走向生产级落地、持续稳定服役的难题。

1.3 Agent Harness (智能体驾驭层)

1.3.1 一句话定义

Agent Harness = Model (大脑) + Harness (神经系统 / 骨架 / 肌肉)。

模型负责推理；Harness 负责执行、控制、安全、可恢复、可审计。Agent Harness 是让 LLM 从“会聊天”变成“能稳定干活”的一整套运行时基础设施与治理机制。

1.3.2 八大核心组件 (工业级标准)

- **编排循环 Orchestration Loop (心跳)**：思考 → 工具 → 观察 → 反思的主循环，控制何时调用工具、何时停止、何时反思。
- **上下文管理 Context Management (防溢出)**：窗口压缩、滑动窗口、摘要、选择性记忆、外部存储，解决长任务上下文爆掉的问题。
- **工具编排 Tool Orchestration (手脚)**：标准化接入（如 MCP 协议）、沙箱隔离、权限控制、调用限流；可控、可审计、可回滚。
- **状态持久化 State Persistence(记忆)**：任务进度快照、断点续跑、历史回溯、Git 式回滚；断了能继续、错了能撤销。
- **安全护栏 & 审批 Safety & HITL (刹车)**：高危操作（删库 / 扣费 / 改配置）人工确认；输入过滤、输出审查、权限最小化。
- **错误处理 & 自动恢复 Error Handling (自愈)**：重试、降级、超时控制、异常捕获、自动回滚；不崩溃、能重试、可兜底。
- **可观测性 Observability (黑盒变白盒)**：执行轨迹日志、决策链追踪、token 统计、耗时分析、失败归因。
- **验证闭环 Verification Loop (质量控制)**：结果校验、单元测试、静态检查、多 Agent 互审、规则引擎。

1.3.3 与相关概念的区别

概念	定位	比喻
Prompt Engineering	一句话怎么写才听懂	话术
Context Engineering	给什么资料才答得准	资料
Agent Framework (LangGraph / AutoGen)	开发时脚手架	写代码用
Agent Harness	生产时操作系统	上线运行用

公式: $Agent = Model + Harness$; *Harness* 决定实际落地效果。

1.3.4 典型 Harness 产品

- LangChain DeepAgents / LangGraph Harness
- Anthropic Claude Code Harness
- OpenAI Agents SDK
- Salesforce Agent Harness
- OpenClaw (本指南主角)

1.3.5 OpenClaw 是 Agent Harness 的典型代表

OpenClaw = 完整的 Agent Harness + 现成的个人助理应用。它不是研究框架，而是把 Harness 所有组件都做好、直接可用的产品。对照八大核心组件，OpenClaw 全部内置实现。

简单类比

- **大模型 (GPT-4o)** = 引擎
- **LangGraph** = 造车图纸
- **OpenClaw** = 已经造好、能直接开的车 (含底盘、刹车、方向盘、仪表盘)

第二章 OpenClaw 是什么

2.1 核心架构（四层解耦）

- **网关层 (Gateway)**：消息路由、模型调度、工具调用管理，支持 50+ 通信平台（微信 / 钉钉 / 飞书等）。
- **智能体层 (Agent)**：对接 GPT-4、Claude、通义千问、Llama 3、Ollama 等，负责意图理解、任务拆解、路径规划。
- **技能层 (Skills)**：模块化“工具集”，内置文件操作、浏览器自动化、命令执行等 50+ 原生技能，支持自定义扩展。
- **记忆层 (Memory)**：持久化上下文，采用向量索引 + Markdown 文件存储，支持跨会话召回与用户偏好记忆。

2.2 核心特性

- **本地部署，安全可控**：全程本地运行，数据不上云，支持 Windows / macOS / Linux / 服务器。
- **模型无关，灵活切换**：兼容所有主流大模型，可自由切换云端 API 或本地模型。
- **多渠道接入**：统一对接 50+ 国内外 IM 与办公平台，跨设备任务不中断。
- **无数据库设计**：采用纯文本（JSONL / Markdown / YAML）存储，可 Git 版本控制，运维成本低。

2.3 典型应用场景

- **办公自动化**：文件整理、报告生成、数据汇总、邮件处理。
- **研发效率**：代码迁移、PR 汇总、自动化测试、文档生成。
- **个人助手**：日程管理、购物下单、健康数据追踪、多平台信息聚合。

2.4 WorkBuddy: OpenClaw 的腾讯发行版

WorkBuddy 是腾讯云 CodeBuddy 团队 2026 年 3 月推出的桌面智能体工作台，基于 OpenClaw 生态做的商业化、免部署版本，内部代号 “小龙虾”，别称 “腾讯龙虾 / QClaw / WorkBuddy”。

一句话理解

OpenClaw 的 “傻瓜化、开箱即用版”：下载安装 → 双击运行 → 一句话指令，直接在 Windows 电脑上干活，不用配环境、不用写代码。

典型场景

- **文件管理**：自动归类、批量改名、清理磁盘。
- **Office 全自动化**：生成 Word / Excel / PPT、数据清洗、图表制作。
- **浏览器自动化**：爬虫、填表、发帖、数据导出。
- **深度集成**：企业微信、QQ、飞书、钉钉，手机远程发指令驱动电脑干活。

和 OpenClaw 的关系（最关键）

- WorkBuddy $\approx$ OpenClaw 的腾讯定制发行版，100% 兼容 OpenClaw 的 Skills（技能包）和 MCP 协议。
- **OpenClaw**：开源、灵活，适合技术人员 / 私有化部署 / 二次开发。
- **WorkBuddy**：免部署、界面友好，适合普通办公 / 企业直接用。

通俗类比：OpenClaw = 自己攒机（自由、便宜、折腾）；WorkBuddy = 品牌整机（开箱即用、省心、优化好）。

2.5 Skill（技能）

2.5.1 一句话定义

Skill = OpenClaw / WorkBuddy 的 “可插拔能力包”，本质是一个带 SKILL.md 的文件夹，用

来教会智能体在什么场景、用什么工具、按什么步骤完成一件事。

2.5.2 Tool vs Skill

- **Tool (工具)**：系统底层能力（读文件、执行命令、调用浏览器），像“手脚”。
- **Skill (技能)**：基于工具封装的完整任务流程（如“整理下载文件夹”、“生成报销表”），像“做好一道菜的菜谱”。

核心特点：免编译、纯文本、可共享、一键安装。

2.5.3 Skill 的目录结构（极简）

一个技能就是一个文件夹，核心只有 SKILL.md，复杂技能可加脚本 / 配置：

```
my-first-skill/  
├ SKILL.md      # 核心：元数据 + 触发条件 + 步骤说明（必须）  
└ scripts/  
  └ main.py     # 可选：Python / Shell 脚本（复杂逻辑用）
```

2.5.4 两种实现方式

方式一：零代码（纯 SKILL.md，新手首选）

只需写一个 SKILL.md，分元数据头（YAML）+ 指令体（Markdown）。

示例：打招呼技能（hello-world / SKILL.md）

```
---  
name: hello-world  
description: 当用户说“打招呼”时，用诗词 + 问候回复  
version: 1.0.0  
author: your-name  
permissions: [] # 无需特殊权限  
---  
  
# 打招呼技能  
  
## 触发条件  
- 用户说：“打招呼”“问候一下”  
  
## 执行步骤
```

1. 输出一句春日诗词（如 “春风十里不如你”）
2. 回复：“你好呀，我是你的智能助手~”
3. 附上今天日期：{{date}}

生效方式：

- 把文件夹放到 ~/.openclaw/workspace/skills/
- 对智能体说：“刷新技能”，自动加载

方式二：代码增强（脚本 + SKILL.md，复杂任务）

适合需要调用 API、处理文件、循环 / 判断的场景（如查天气、自动化 Excel）。

示例：天气查询技能 weather-query/

```
weather-query/  
├ SKILL.md  
└ scripts/  
  └ main.py
```

SKILL.md（声明触发与调用）：

```
---  
name: weather-query  
description: 查询城市实时天气，返回温度 / 湿度 / 风力  
permissions: ["network"]  
---  
  
# 天气查询技能  
  
## 触发  
- “北京今天天气” “上海温度”  
  
## 执行  
调用脚本: python scripts/main.py {{city}}
```

scripts/main.py（核心逻辑）：

```
import sys, requests  
city = sys.argv[1]  
res = requests.get(f"https://wttr.in/{city}?format=j1").json()  
print(f"{city}: {res['current_condition'][0]['temp_C']}°C, 湿度  
{res['current_condition'][0]['humidity']}%")
```

2.5.5 Skill 工作流（智能体怎么用它）

1. 扫描加载：启动 / 刷新时，扫描 skills/ 目录，读取所有 SKILL.md。

2. 意图匹配：用户发指令 → 大模型理解意图 → 匹配 description 和触发条件。
3. 执行调用：匹配成功 → 按 SKILL.md 步骤执行（调用工具 / 脚本）。
4. 返回结果：把执行结果整理成自然语言回复用户。

2.5.6 常用技能来源

- **ClawHub (官方市场)**：数千个现成技能（文件整理、Excel 自动化、邮件处理、爬虫），一键安装。
- **社区共享**：GitHub / Gitee 搜索 openclaw-skill，直接下载使用。
- **WorkBuddy 内置**：腾讯预装办公常用技能（微信 / QQ / 企业微信集成）。

2.6 MCP (Model Context Protocol)

2.6.1 一句话定义

MCP = Model Context Protocol（模型上下文协议），由 Anthropic 于 2024 年底推出的开放标准，俗称“AI 的 USB-C 接口”：让大模型 / 智能体用一套统一方式安全连接任意外部工具、数据和服务。

2.6.2 解决什么问题

以前各 AI 平台（Claude、OpenAI、OpenClaw）各有私有插件 / Function Calling，互不兼容，工具要重复开发。

MCP 做到：一次开发（MCP Server），所有 MCP 客户端都能用（Claude Desktop、OpenClaw、WorkBuddy、Cursor 等）。

2.6.3 核心角色 (Client ↔ Server)

- **MCP Client (客户端)**：OpenClaw / WorkBuddy / Claude Desktop，负责发起调用、接收结果、交给大模型决策。

- **MCP Server (服务端)**：独立进程，封装一个或多个工具能力（如查天气、操作 Excel、连数据库、控制浏览器），对外暴露标准接口。

2.6.4 MCP 提供的三类能力

- **Tools (工具)**：可调用函数（如“创建 PPT”、“查订单”）。
- **Resources (资源)**：可读取数据（文件、数据库、API 数据）。
- **Prompts (提示模板)**：可复用任务模板（如“写周报”、“数据清洗”）。

2.6.5 Skill vs MCP (别混淆)

维度	Skill	MCP
归属	OpenClaw 内部能力包	外部开放标准协议
实现	SKILL.md + 脚本	独立 MCP Server 服务
复用范围	仅 OpenClaw / WorkBuddy 可用	跨平台通用 (Claude / Cursor / OpenClaw 等都能调)
典型例子	excel-generator 技能，只能在 OpenClaw 用	excel-mcp 服务，多个 AI 客户端都能调用

2.6.6 MCP 工作流程 (极简)

1. 在 OpenClaw 配置一个 MCP Server (填地址、认证)。
2. OpenClaw (Client) 连接 MCP Server，自动发现它的 Tools / Resources。
3. 用户发指令：“帮我把今天销售数据生成 Excel 并发企业微信”。
4. Agent 决策：调用 MCP 的“数据库查询” → “Excel 生成” → “企业微信发送”三个工具。
5. MCP Server 执行并返回结果，Agent 整理成自然语言回复。

一句话总结：MCP 是 AI 的通用扩展总线——标准化、安全隔离、跨平台复用；OpenClaw 通过 MCP 把能力从本地文件 / 命令，扩展到连接一切在线服务与设备。

第三章 OpenClaw 工作原理与流程

3.1 五步固定工作流程

任何任务都走这 5 步：

1. **意图理解**：读懂自然语言指令。
2. **任务拆解**：把大任务拆成多步小任务。
3. **技能调度**：自动选浏览器 / 文件 / 命令 / Excel 等工具。
4. **循环执行 + 自我纠错**：一步一步做，出错自动重试。
5. **结果汇总 + 本地存档**：整理报告、保存文件、生成摘要。

3.2 三个真实工作过程实例

3.2.1 案例一：办公场景——行业资料整理

用户指令

帮我去百度搜 “2026 人工智能智能体行业报告”，把前 5 篇网页核心内容提炼，整理成 Excel 表格，保存到桌面「AI 行业」文件夹。

工作全过程

- **意图理解**：识别任务为网页搜索 → 内容抓取 → 文本提炼 → 生成 Excel → 指定文件夹保存。

任务自动拆解：

- 调用浏览器技能，打开百度搜索关键词
- 抓取前 5 篇官网 / 行业报告网页正文
- 每篇提炼：标题、发布机构、核心观点、关键数据
- 调用表格生成技能，按固定表头建 Excel

- 检测桌面有没有「AI 行业」文件夹，没有就自动新建
- 把 Excel 移入对应文件夹，生成任务摘要

输出结果：桌面生成「AI 行业」文件夹 → 规整的汇总 Excel → 一份文字版要点总结。

3.2.2 案例二：研发场景——代码项目自动化整理

用户指令

把我 D 盘 project 文件夹下所有 Python 代码，统一格式化，找出所有 print 调试语句并标注，生成一份代码检查报告 md 文件。

拆解步骤

1. 调用本地文件遍历技能：扫描 D 盘 project 下所有 .py 文件
2. 调用代码格式化技能：批量统一缩进、规范格式
3. 逐文件分析：匹配所有 print 调试代码，记录所在行号
4. 自动创建 代码检查报告.md
5. 汇总问题列表、文件路径、待优化点写入文档

最终输出：格式化后的源码 + 一份完整检查报告，全程自动执行、无需打开 IDE。

3.2.3 案例三：个人日常自动化

用户指令

把我下载文件夹里所有 PDF，按“学术 / 合同 / 杂项”自动分类归档，每个文件夹生成内容摘要。

拆解执行

1. 遍历 Download 文件夹，筛选全部 PDF
2. 调用 PDF 解析技能，读取每篇正文标题、关键字
3. 智能判断归属：论文 → 学术、协议 → 合同、其他 → 杂项

4. 自动在 Download 下新建三个分类文件夹
5. 自动移动文件、逐个生成简短摘要保存到目录说明里

3.2.4 从实例看本质特点

- **不用写 workflow**：自然语言直接下达复杂多步任务，自己拆步骤。
- **自带技能库**：浏览器、文件、Excel、PDF、代码、命令行随取随用。
- **本地全权操作**：直接读写电脑文件、建文件夹、生成文档。
- **能自我纠错**：网页打不开、文件夹不存在，会自动重试、自动新建。
- **无代码门槛**：普通人一句话就能做原本几小时的工作。

3.3 端到端串讲：用户指令 → Agent → MCP → Skill → Tool

用一个真实例子串讲全流程。

任务场景：帮我把本月发票文件夹里所有 PDF 发票，提取金额、日期、开票方，整理成 Excel，再发到企业微信工作群。

3.3.1 角色对应

- **LLM 大模型**：大脑，负责思考。
- **Agent 智能体**：总指挥，拆任务、做决策。
- **Skill 技能**：预设好的「发票整理全套流程」。
- **MCP 协议**：连接 PDF 解析、Excel、企业微信的标准通道。
- **Tool 工具**：读文件、解析 PDF、生成表格、发消息。

3.3.2 完整工作流程（7 步走）

第 1 步：用户发指令

“把本月发票 PDF 提取信息，整理 Excel，发企业微信群”



第 2 步：Agent 理解 + 任务规划

大模型思考拆解成 4 步：遍历文件夹找 PDF → 解析每张发票 → 生成规范 Excel → 自动发送到企业微信群。

第 3 步：匹配本地 Skill 技能

OpenClaw 扫描内置发票处理 Skill，里面写好了：触发关键词、可用工具、执行规则、表头规范。

第 4 步：通过 MCP 调用外部能力

OpenClaw 作为 MCP Client，去连接三个 MCP Server：PDF 解析 MCP、Excel 生成 MCP、企业微信机器人 MCP。

第 5 步：调用底层 Tool 工具执行

- **文件工具**：遍历文件夹，筛选所有 PDF。
- **PDF 工具**：读取每页内容，提取金额 / 日期 / 开票方。
- **表格工具**：新建 Excel，按行列填入数据。
- **消息工具**：登录企业微信，上传文件并发群消息。

第 6 步：中间自我校验 (Reflection)

智能体自动检查：有没有漏发票？格式对不对？金额有没有异常？有错就自动重试、补抓、修正。

第 7 步：完成并反馈

生成好的发票汇总 Excel 保存在本地，自动发到企业微信群，给用户返回一句话总结：共处理 XX 张发票，已归档并推送完毕。

第四章 OpenClaw 开发指南

4.1 你能在 OpenClaw 里开发什么？

六大开发方向

- **自定义智能体人设 / 规则 / 提示词**：定角色（行政、财务、助教、分析师）、做事规则、输出格式。
- **开发自定义技能 (Skill)**：把重复工作做成一句话触发的自动化能力，如 Excel 自动统计、文档批量整理、周报自动生成。
- **开发 MCP 服务 (最核心、最值钱)**：自己写工具 / 接口，让 OpenClaw 能调用，连接数据库、企业 API、内部系统、自定义业务功能。
- **多智能体协作开发**：创建多个角色智能体，如调度 Agent、执行 Agent、审核 Agent、总结 Agent。
- **上下文 / 记忆策略开发**：自定义记忆保留长度、自动摘要、敏感信息脱敏、历史对话管理。
- **流程编排 (类似 LangGraph)**：开发带判断、循环、分支的复杂任务流程。

4.2 两种开发模式选型

- **轻量开发**：只写 Skill 插件，不用改框架源码，适合做自动化、机器人、小工具（90% 人用这种）。
- **深度二次开发**：改内核架构、自定义记忆逻辑、改对话流程，适合做企业私有化定制。

4.3 Skill 插件开发详解（核心开发入口）

所有自定义功能，全部以 Skill 形式开发，这是 OpenClaw 唯一开发入口。

4.3.1 技能文件位置与结构

路径: ./skills/你的自定义技能文件夹/

每个技能就是一个独立文件夹，里面固定 2 个文件：

- **skill.json**: 技能配置、描述、触发指令。
- **index.js / index.ts**: 业务逻辑代码。

4.3.2 最简开发模板（直接复制可用）

① skill.json（配置文件）

```
{
  "name": "我的自定义工具",
  "description": "可以帮我计算、查信息、执行自定义任务",
  "triggers": ["计算", "帮我执行", "查数据"],
  "enabled": true
}
```

② index.js（逻辑代码）

```
// 入口函数
async function run(context) {
  // context.userInput 用户输入
  const msg = context.userInput;
  // 在这里写你任何自定义逻辑
  let res = "我已收到指令: " + msg;
  // 返回给 AI 回复
  return res;
}

module.exports = { run };
```

写完保存，不用重启框架，OpenClaw 自动加载新技能。

4.3.3 常见逻辑写法

- **调用第三方 API**: 在 run 函数里用 axios 发请求，对接天气、爬虫、企业接口、数据库。
- **本地文件操作**: 读写 txt / excel、整理文件夹、生成报告文档。
- **执行系统命令**: 调用 shell / python 脚本、运维操作、自动化任务。



- **接入私有业务系统：**对接公司内网接口、查订单、查库存、查考勤。

第五章 OpenClaw 文件结构详解

5.1 总览：所有开发结果存在哪里？

全部统一放在一个文件夹（Windows）：

```
C:\Users\你的用户名\.openclaw
```

下面按目录逐一说明：是什么、放什么、长什么样。

5.2 workspace 文件夹（最重要）

路径：.openclaw\workspace

核心文件

SOUL.md：智能体的人设、性格、规则、提示词

内容示例：

```
你是企业行政办公智能体。  
只写通知、公文、制度。  
语气正式、不闲聊、不编造。
```

USER.md：你的个人信息、习惯、偏好

```
姓名：张三  
部门：行政部  
常用模板：公司通知模板
```

AGENTS.md：多智能体角色定义

```
调度 Agent：负责拆任务  
执行 Agent：负责处理文档  
审核 Agent：负责检查错误
```

这个文件夹还存放：要处理的 Excel / PDF / 文档、智能体生成的报告与输出文件。

5.3 skills 文件夹（自定义技能）

路径：.openclaw\skills

结构（每个技能 = 一个文件夹 + 一个 SKILL.md）：

```
skills/
```

```
├── 日报生成/
│   └── SKILL.md
├── Excel 清洗/
│   └── SKILL.md
└── PDF 合并/
    └── SKILL.md
```

SKILL.md 固定格式示例:

名称: 日报生成
描述: 自动读取今日工作, 生成标准日报
触发词: 帮我生成日报
步骤:
1. 读取 workspace/today.md
2. 按格式整理
3. 保存到 workspace/日报.md

5.4 agents 文件夹 (多智能体配置)

路径: .openclaw\agents

结构 (每个智能体一个文件夹) :

```
agents/
├── 调度助手/
│   └── config.json
├── 文档处理/
│   └── config.json
└── 审核专家/
    └── config.json
```

config.json 内容示例:

```
{
  "name": "文档处理助手",
  "description": "专门处理 Word、Excel",
  "model": "deepseek",
  "temperature": 0.1
}
```

5.5 mcp 文件夹 (自定义 MCP 工具)

路径: .openclaw\mcp

```
mcp/
├── mysql/
│   └── server.py
├── excel 工具/
│   └── server.py
└── 企业 API/
```

MCP 是你写的 Python 工具，让智能体能调用数据库、API、自定义功能。

5.6 openclaw.json（全局配置文件）

路径：.openclaw\openclaw.json

作用：配置模型、路径、网关。

```
{  
  "model": "deepseek",  
  "api_key": "sk-xxxx",  
  "workspace": ".openclaw/workspace",  
  "max_memory_rounds": 10,  
  "mcp_servers": ["mysql", "excel 工具"]  
}
```

5.7 logs 文件夹（日志）

路径：.openclaw\logs。存放运行日志，用于排查错误。

第六章 OpenClaw 配置详解

6.1 配置总览

OpenClaw 所有配置都集中在 `~/.openclaw/` 目录，主配置是 `openclaw.json` (JSON5 格式，支持注释)。

日常改得最多的是：模型、Agent 行为、渠道、技能 / 工具权限、记忆 / 上下文、工作路径。

6.2 主配置文件位置与查看方式

文件： `~/.openclaw/openclaw.json`

查看全部配置：

```
openclaw config get
```

查看某项：

```
openclaw config get agents.defaults.temperature
```

修改某项：

```
openclaw config set agents.defaults.temperature 0.7
```

6.3 最常改的 6 大类配置

6.3.1 模型配置 (必改)

```
{
  "agents": {
    "defaults": {
      "model": {
        "primary": "ollama/qwen2.5:7b", // 默认模型
        "fallbacks": ["openai/gpt-3.5-turbo"] // 降级备用
      },
      "temperature": 0.7, // 随机性 0~1
      "maxTokens": 16000, // 最大上下文
      "workspace": "~/.openclaw/workspace"
    }
  },
  "models": {
    "providers": {
      "openai": {
        "apiKey": "sk-xxx",
```

```
"baseUrl": "https://api.openai.com/v1"
},
"ollama": {
  "baseUrl": "http://localhost:11434"
}
}
}
```

可改项：

- **primary / fallbacks**: 换模型（GPT、Claude、通义、本地 Ollama）。
- **temperature**: 回答更 creative (1) 或更稳定 (0) 。
- **maxTokens**: 上下文长度，越长越“记东西”。
- **apiKey / baseUrl**: 换模型服务商 / 私有部署地址。

6.3.2 Agent 行为与记忆

```
"agents": {
  "defaults": {
    "memory": {
      "enabled": true,
      "maxHistory": 20,           // 记忆多少轮对话
      "compressThreshold": 8000  // 超过多少 token 自动压缩
    },
    "personality": {
      "name": "小助手",
      "style": "专业简洁",
      "language": "zh-CN"
    }
  }
}
```

可改项：

- **maxHistory**: 想让它记久一点就调大。
- **compressThreshold**: 长对话防爆 token。
- **personality**: 改名字、语气、语言。

另外 workspace 里还有 4 个“人格 / 工作手册”文件，纯文本直接改：

- **soul.md**: 性格设定。
- **agents.md**: 工作流程、安全边界。

- **user.md**: 你的信息。
- **identity.md**: 头像、昵称。

6.3.3 渠道配置 (微信 / 钉钉 / Telegram 等)

```
"channels": {
  "defaults": {
    "enabled": false,
    "responseTimeout": 60
  },
  "wechat": {
    "enabled": true,
    "botToken": "xxx",
    "allowFrom": ["user1", "user2"]
  },
  "dingtalk": {
    "enabled": true,
    "appId": "xxx",
    "appSecret": "xxx"
  }
}
```

可改项：开关渠道、填机器人 token / 密钥、限制谁能私聊 (allowFrom) 、超时时间、重试次数。

6.3.4 技能 / 工具权限

```
"commands": {
  "native": "allow",      // 系统命令执行: allow / deny / auto
  "nativeSkills": "allow",
  "restart": true
},
"sandbox": {
  "enabled": true,
  "allowWrite": true,     // 是否允许写文件
  "allowNetwork": true   // 是否允许联网
}
```

可改项：

- **native: deny**: 禁止执行系统命令 (安全加固) 。
- **sandbox**: 控制文件读写、网络访问。

6.3.5 文件与工作路径

```
"agents": {
  "defaults": {
    "workspace": "/data/openclaw/workspace"
  }
}
```



```
}  
},  
"storage": {  
  "memoryDir": "~/openclaw/memory",  
  "cacheDir": "~/openclaw/cache"  
}  
}
```

可改项：把数据移到更大磁盘，备份 / 迁移时直接改路径。

6.3.6 性能与安全（高级）

```
"gateway": {  
  "port": 8080,  
  "maxConcurrent": 10,  
  "cors": true  
},  
"security": {  
  "apiKeyRequired": true,  
  "rateLimit": {  
    "enabled": true,  
    "maxRequests": 100,  
    "windowMs": 60000  
  }  
}  
}
```

可改项：改端口、最大并发；开启 / 关闭 API 密钥验证；限流防刷。

6.4 日常修改建议顺序

1. **先改** 模型 + API Key（跑起来）。
2. **再改** 渠道（微信 / 钉钉能用）。
3. **调整** memory.maxHistory + compressThreshold（长对话稳定）。
4. **改** personality + soul.md / agents.md（性格和工作流程）。
5. **收紧** sandbox + commands.native（安全）。

6.5 Agent Harness 八大组件在 OpenClaw 中的实现

OpenClaw 把第一章介绍的八大 Harness 组件全部内置实现，每个组件都有对应的源码位置与定制入口。

组件	核心源码位置	最常用定制方式
编排循环	src/agent/agent-loop.ts	改 AGENTS.md 思考规则
上下文 / 记忆	src/memory/ + MEMORY.md	调整压缩策略、新增记忆文件
工具编排	src/skills/ + tool-policy.ts	openclaw tool create 建新工具
状态持久化	src/storage/ + workspace/sessions	改存储格式、加自动备份
安全护栏	src/security/ + guardrails.ts	调整审批级别、新增黑名单
错误恢复	src/utils/retry.ts + error- handler.ts	改重试次数、自定义降级逻辑
可观测性	src/log/ + src/monitor/metrics.ts	调日志级别、新增业务指标
验证闭环	src/validator/ + result- checker.ts	新增校验规则、改通过标准

第七章 行业实践：财务垂直行业智能体开发

7.1 开发总体说明

本财务垂直智能体不修改系统内核源码，采用：新增自定义 Skills + 修改行业配置文件 + 少量修改安全规则。下文涉及的所有文件，原版 OpenClaw 均不存在，全部由开发者二次开发实现。

开发分为四类文件：

- **人设约束文件（普通用户可改）**：AGENTS.md、SOUL.md。
- **财务技能文件（开发者新增）**：skills/finance 全套。
- **行业 workflow 文件（开发者新增）**：财务流程 JSON。
- **系统适配文件（少量修改）**：guardrails.ts、日志、持久化配置。

7.2 第一类：工作区人设文件（必须改）

7.2.1 workspace/AGENTS.md（财务智能体人设）

```
# 身份
你是企业专业财务智能顾问，精通企业会计准则、增值税、企业所得税、费用报销规范。

# 能力
1、解析 Excel 财务报表、利润表、费用表
2、识别电子发票、校验发票合规性
3、自动整理财务文件、归档分类
4、税务政策查询、涉税风险筛查
5、生成月度财务分析简报

# 输出规范
1、金额统一使用人民币，保留两位小数
2、风险数据必须标红提醒
3、报表分析必须包含：环比、成本、费用、风险

# 禁止行为
1、不得提供虚开发票、违规避税方案
2、不得篡改原始财务数据
3、大额资金必须提示人工复核
```

7.2.2 workspace/SOUL.md (财务行为约束)

```
# 行为准则
1、严格遵守国家财税法律，拒绝一切违规财务操作
2、所有财务分析留痕、可审计
3、原始单据禁止删除、禁止修改
4、单笔金额大于 20000 元必须二次确认
5、生成报告自带免责声明：仅供内部财务参考，不构成法律依据

# 工作风格
严谨、保守、真实、不夸大、不随意预判财务走势
```

7.3 第二类：财务自定义 Skills (核心开发)

路径：src/skills/custom/finance/（原版没有，全部新建）。

7.3.1 skills/custom/finance/SKILL.md (技能说明)

```
# 财务分析工具

## 触发条件
上传 Excel 报表、发票、费用单据、财务压缩包

## 功能
1、自动清洗财务表格，修复破损单元格
2、统计收入、成本、费用、净利润
3、识别异常金额、负数金额、不合理报销
4、生成财务月报、费用饼图、收支汇总
5、自动归档财务文件

## 限制
不可修改原始凭证，大额支出自动风控拦截
```

7.3.2 skills/custom/finance/finance-tool.ts (财务主工具代码)

```
// 财务行业自定义工具（开发者新增）
import { readExcel, saveFile, scanInvoice } from '../utils'

export default {
  name: "finance_analysis_tool",
  description: "财务报表解析、发票识别、费用统计、自动归档",
  async run(filePath: string) {
    // 1、读取财务表格
    const tableData = await readExcel(filePath);
    // 2、统计收支、成本、费用
    let income = tableData.income;
```

```
let cost = tableData.cost;
let profit = income - cost;

// 3、财务风控判断
let riskMsg = "";
if (income > 500000) {
  riskMsg = "本月收入偏高，建议核查发票流向";
}
if (cost > income) {
  riskMsg += "⚠️ 本月亏损，成本异常偏高";
}

// 4、自动归档文件
await saveFile(filePath, "./workspace/finance_record/");

// 5、返回财务简报
return {
  本月总收入: income,
  本月总成本: cost,
  本月净利润: profit,
  风险提示: riskMsg,
  处理状态: "财务分析完成，已自动归档"
}
}
```

7.3.3 skills/custom/finance/rules.md (财务风控规则)

```
# 财务风控规则
1、单笔报销金额大于 20000 元，禁止自动通过
2、负数金额、异常红字自动标红
3、餐饮费用单月超过总收入 15% 判定异常
4、发票抬头错误、税号错误自动拦截
5、禁止生成任何避税、虚开发票话术
```

7.4 第三类：财务 workflow 文件（新增）

路径：src/planner/workflow-templates/finance-monthly.json

```
{
  "name": "企业月末财务复盘流程",
  "description": "标准化财务月度审核流程",
  "steps": [
    "上传本月财务总账报表",
    "智能清洗 Excel 破损数据",
    "校验借贷金额是否平衡",
    "统计收入、成本、费用",
    "环比上月财务数据",
  ]
}
```

```
"筛查涉税风险、费用异常",  
"生成财务月度报告",  
"自动归档保存财务凭证"  
]  
}
```

7.5 第四类：系统少量修改文件

原版系统文件，少量修改以适配财务行业。

7.5.1 修改：src/security/guardrails.ts (财务风控)

```
// 新增财务行业风控代码  
export function checkFinanceRisk(text: string, money: number) {  
  // 大额资金风控  
  if (money > 20000) {  
    return { risk: true, msg: "金额超过 20000 元，需人工财务复核" }  
  }  
  // 违规财税话术拦截  
  if (text.includes("避税") || text.includes("虚开发票")) {  
    return { risk: true, msg: "禁止违规财税操作建议" }  
  }  
  return { risk: false }  
}
```

7.5.2 修改：src/log/audit.ts (财务日志)

```
// 新增财务专属日志字段  
const financeLog = {  
  userName: ctx.user,  
  userIP: ctx.ip,  
  fileName: ctx.fileName,  
  moneyRange: ctx.money,  
  operateTime: new Date()  
};
```

7.5.3 修改：src/utils/retry.ts (Excel 容错)

```
// 财务表格容错：破损单元格自动跳过  
if (cell.isError || cell.isEmpty) {  
  continue;  
}
```

7.6 文件分类总结

7.6.1 普通用户可修改文件（无代码）

- **workspace/AGENTS.md**：修改财务人设、回答规范。

- **workspace/SOUL.md**: 修改财务约束、禁止行为。
- **skills/finance/SKILL.md**: 调整财务技能触发方式。

7.6.2 开发者新增文件（必须写代码）

- **finance-tool.ts**: 财务核心解析工具。
- **rules.md**: 财务风控规则文档。
- **finance-monthly.json**: 财务月末工作流。

7.6.3 开发者修改系统文件（少量改动）

- **guardrails.ts**: 增加金额风控。
- **audit.ts**: 增加财务审计日志。
- **retry.ts**: 优化财务 Excel 容错。

7.7 一句话总结

本次基于 OpenClaw 开发财务垂直行业智能体，不改动系统内核源码。普通用户通过修改 AGENTS.md、SOUL.md 完成财务人设配置；开发者新增财务 Skills 工具、财务工作流模板，并少量修改安全护栏、审计日志、容错适配文件，实现财务报表解析、发票审核、资金风控、自动归档等行业能力，满足中小企业财务智能化、合规化、私有化部署需求。